

GitPub

System Design Document

A modern, self-hosted Git Version Control System & CI/CD Sandbox Platform
built with microservice architecture.

Go

Node.js

Next.js

PostgreSQL

Redis

Docker

BullMQ

Interview-Ready • Comprehensive • Visual
Generated September 2026



Table of Contents

1. [Project Description](#)
2. [Functional Requirements](#)
3. [Non-Functional Requirements](#)
 1. Scalability
 2. CAP Theorem Analysis
 3. Latency Targets
 4. Reliability & Fault Tolerance
 5. Security
4. [Core Entities \(ER Diagram\)](#)
5. [API Design \(Mapped to FRs\)](#)
6. [High-Level Design \(HLD\)](#)
 1. System Architecture
 2. Git Push — End-to-End Sequence
 3. Pull Request Merge Flow
7. [Low-Level Design \(LLD\)](#)
 1. Authentication System
 2. Git Protocol Gateway
 3. CI Pipeline Execution
 4. PR Diff & Merge Engine
8. [Database Access Patterns & Indexing](#)
9. [Scaling Deep Dive \(10x / 100x\)](#)
10. [Production Deployment Architecture](#)
11. [Interview Cheat Sheet](#)

1 Project Description

GitPub is a modern, self-hosted **Git version control system** and **CI/CD sandbox platform** — essentially a simplified, self-hostable alternative to GitHub + GitHub Actions.

What does it do?

CAPABILITY	DESCRIPTION
Git Hosting	Host Git repos on your own server. Clone, push, fetch using standard Git CLI over HTTP Smart Protocol.
Authentication	Multi-strategy auth: email/password (Argon2id), Google OAuth 2.0, and PATs for Git CLI.
Repo Management	Create public/private repos, browse file trees, view file contents via web dashboard.
Pull Requests	Open PRs between branches, view unified diffs, merge with conflict detection.
CI/CD Pipelines	Auto-build & test code on every <code>git push</code> inside hardened Docker sandbox containers. Live-stream logs.
Web Dashboard	Next.js Bento Grid UI with Monaco Editor, diff viewer, and real-time CI console.

Why is this interesting from a system design perspective?

- **Streaming I/O:** Git packfiles can be 50MB–1GB+. The system uses `io.Pipe` for zero-RAM buffering.
- **Multi-language microservices:** Go gateway (performance), Node.js API (productivity), Next.js frontend.
- **Sandboxed execution:** Running untrusted user code safely with Linux capability dropping, network isolation, memory limits.
- **Event-driven architecture:** Push events → message queue → async CI workers.

2 Functional Requirements

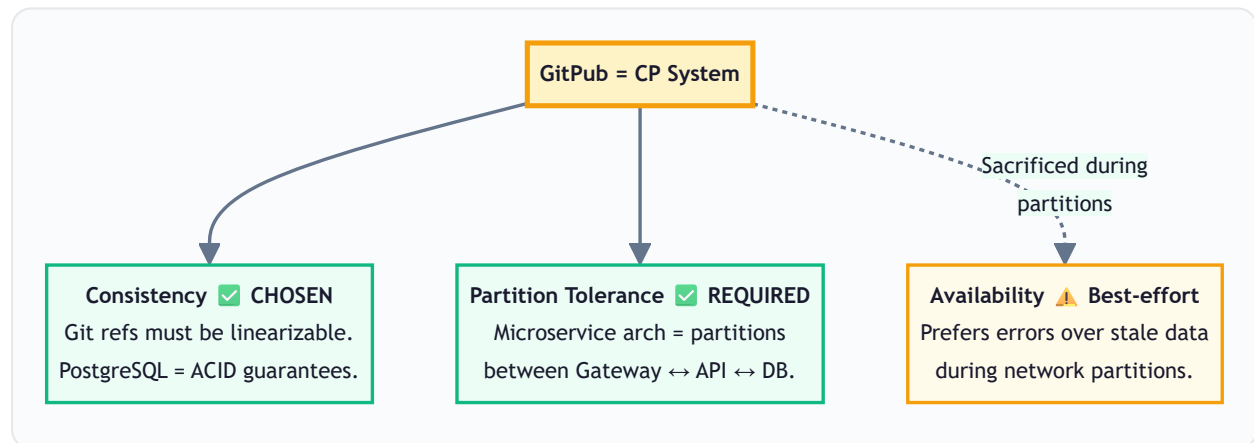
ID	REQUIREMENT	CATEGORY
FR-1	Users can register, login (email/password), and authenticate via Google OAuth	Auth
FR-2	Users can generate Personal Access Tokens (PAT) for Git CLI authentication	Auth
FR-3	Users can create public/private Git repositories	Repo Management
FR-4	Users can clone, push, and fetch repos using standard Git CLI	Git Protocol
FR-5	Users can browse repo file trees and view file contents via web UI	Repo Browsing
FR-6	Users can open Pull Requests between branches	PR Engine
FR-7	Users can view unified diffs for Pull Requests	PR Engine
FR-8	Repository owners can merge Pull Requests (with conflict detection)	PR Engine
FR-9	CI pipelines are automatically triggered on <code>git push</code>	CI/CD
FR-10	Users can manually trigger CI runs on any branch/commit	CI/CD
FR-11	CI logs are streamed in real-time to the database and UI	CI/CD
FR-12	CI runs execute inside hardened, sandboxed Docker containers	CI/CD

3 Non-Functional Requirements

3.1 Scalability

ASPECT	CURRENT DESIGN	SCALE STRATEGY
Read-heavy traffic (clones/fetches)	Single Go gateway	Horizontal scaling: stateless Go gateway behind a load balancer. Repos on shared NFS/EFS.
Write traffic (pushes)	Single writer	Shard repos by owner hash across storage nodes. Consistent hashing.
CI job throughput	Single BullMQ worker	Scale workers horizontally — each is stateless and pulls from Redis queue.
Database	Single PostgreSQL	Read replicas for dashboard. Vertical scaling for writes. Partition <code>ci_runs</code> by <code>created_at</code> .

3.2 CAP Theorem Analysis



Verdict: CP system. *Git is inherently a CP system — refs must be consistent. We sacrifice availability during partitions rather than risk data corruption.*

3.3 Latency Targets

OPERATION	TARGET	HOW ACHIEVED
Git clone (small repo)	< 500ms	<code>io.Pipe</code> zero-copy streaming. Go goroutine-per-connection.
Git clone (500MB repo)	Stream-dependent	O(1) memory via <code>io.Pipe</code> . Latency = network bandwidth.
API CRUD operations	< 100ms	PostgreSQL connection pooling (<code>pg.Pool</code>), indexed queries.
CI job start	< 5s	BullMQ near-instant dequeue. Docker cold start ~2-3s.
CI log polling	5s intervals	Frontend polls <code>/api/ci/:id</code> . Could upgrade to WebSocket/SSE.
PR diff computation	< 2s	Native <code>git diff</code> on bare repo — no clone needed.

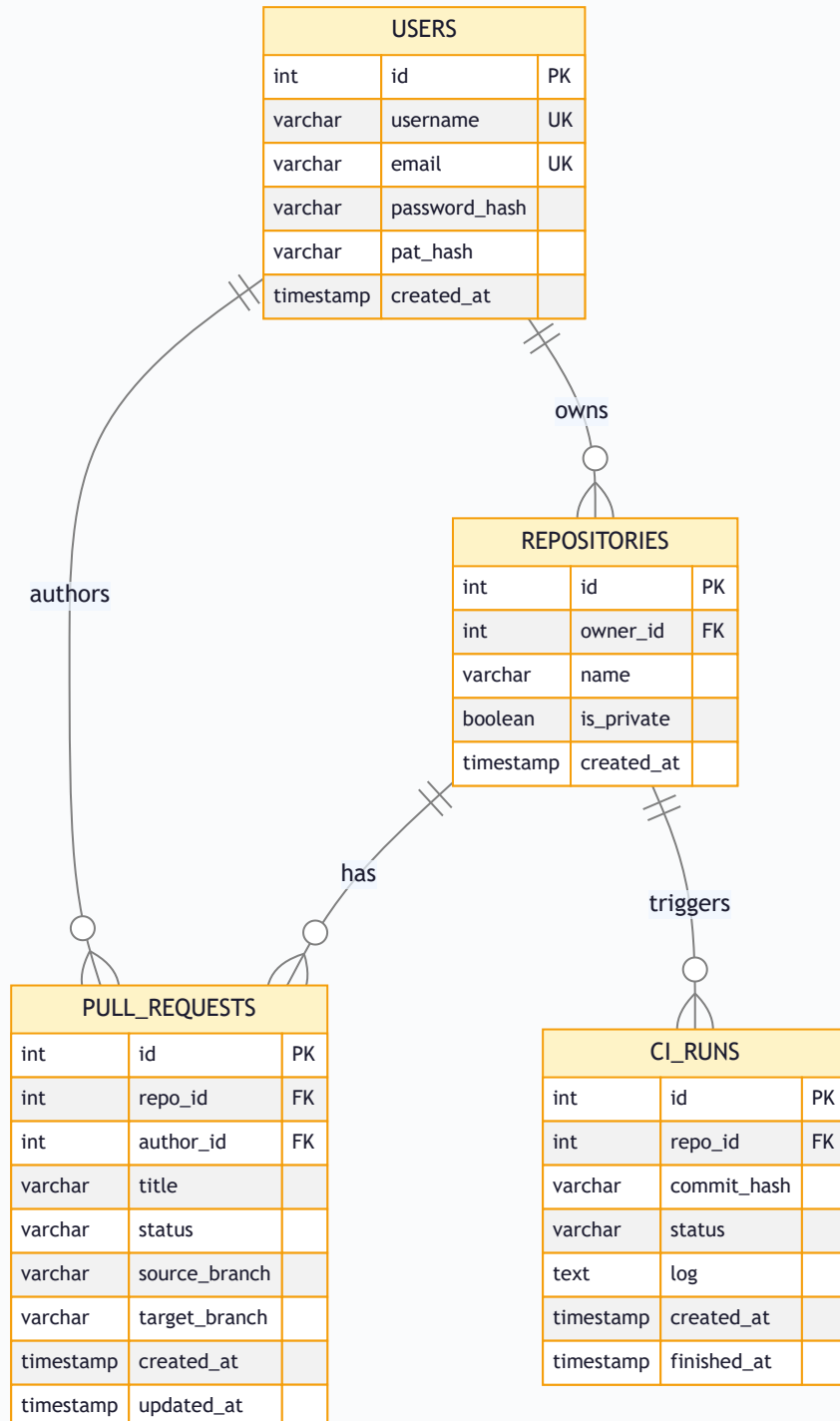
3.4 Reliability & Fault Tolerance

CONCERN	MITIGATION
CI runner crash	10-minute watchdog timeout with <code>SIGKILL</code> . Temp workspace cleaned in <code>finally</code> block.
Docker unavailable	Graceful fallback to local subprocess execution (dev mode only).
Database failure	Health check endpoint pings DB. Container orchestrator restarts on failure.
Orphaned containers	<code>--rm</code> flag auto-destroys containers on exit.

3.5 Security

LAYER	MECHANISM
Password storage	Argon2id (memory-hard, side-channel resistant)
Token storage	SHA-256 hashed PATs — plaintext never stored
JWT	HMAC-SHA256, 24h expiry
CI Sandbox	<code>--cap-drop=ALL</code> , <code>--no-new-privileges</code> , <code>--network none</code> , <code>--memory=1024m</code> , <code>--cpus=1.0</code> , non-root user
RBAC	Owner-only push, owner-only merge, private repo access control

4 Core Entities



Entity Relationships

RELATIONSHIP	CARDINALITY	CASCADE
User → Repository	1 : N	ON DELETE CASCADE
User → Pull Request	1 : N	ON DELETE CASCADE
Repository → Pull Request	1 : N	ON DELETE CASCADE
Repository → CI Run	1 : N	ON DELETE CASCADE

Status Enums

ENTITY	FIELD	VALUES
pull_requests	status	open → merged
ci_runs	status	pending → running → success failed



5

API Design (Mapped to Functional Requirements)

5.1 Authentication APIs → FR-1, FR-2

METHOD	ENDPOINT	AUTH	MAPS TO	DESCRIPTION
POST	/api/auth/register	None	FR-1	Register with <code>{ username, email, password }</code> . Argon2id hash.
POST	/api/auth/login	None	FR-1	Login. Returns 24h JWT <code>{ token }</code> .
POST	/api/auth/google	None	FR-1	Google OAuth. <code>{ credential }</code> ID token. Auto-creates user.
POST	/api/auth/pat	JWT	FR-2	Generate PAT <code>gp_pat_<48hex></code> . SHA-256 stored. Plaintext returned once.

5.2 Repository APIs → FR-3, FR-5

METHOD	ENDPOINT	AUTH	MAPS TO	DESCRIPTION
POST	/api/repos	JWT	FR-3	Create repo. Runs <code>git init --bare</code> . Returns clone URL.
GET	/api/repos	JWT	FR-3	List user's repos + public repos.
GET	/api/repos/:owner/:repo/files	JWT	FR-5	Browse file tree via <code>git ls-tree</code> .
GET	/api/repos/:owner/:repo/file-content	JWT	FR-5	View file via <code>git show</code> . Query: <code>?path=...</code>

5.3 Git Protocol APIs → FR-4 (Go Gateway)

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	<code>/:owner/:repo.git/info/refs?service=git-upload-pack</code>	Basic	Fetch/clone ref discovery
GET	<code>/:owner/:repo.git/info/refs?service=git-receive-pack</code>	Basic	Push ref discovery
POST	<code>/:owner/:repo.git/git-upload-pack</code>	Basic	Clone/fetch packfile stream
POST	<code>/:owner/:repo.git/git-receive-pack</code>	Basic	Push packfile stream + trigger CI

5.4 Internal Verification API

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	<code>/api/repos/verify-git-auth</code>	Internal	Gateway verifies PAT credentials. Returns <code>{ authenticated, authorized }</code> .

5.5 Pull Request APIs → FR-6, FR-7, FR-8

METHOD	ENDPOINT	AUTH	MAPS TO	DESCRIPTION
POST	<code>/api/pulls</code>	JWT	FR-6	Create PR <code>{ repoId, title, sourceBranch, targetBranch }</code> .
GET	<code>/api/pulls?repoId=X</code>	JWT	FR-6	List PRs for a repository with author names.
GET	<code>/api/pulls/:id/diff</code>	JWT	FR-7	Compute diff via <code>git diff "target...source"</code> .
POST	<code>/api/pulls/:id/merge</code>	JWT	FR-8	Merge PR: temp clone → merge → push back. Owner-only.

5.6 CI/CD APIs → FR-9, FR-10, FR-11

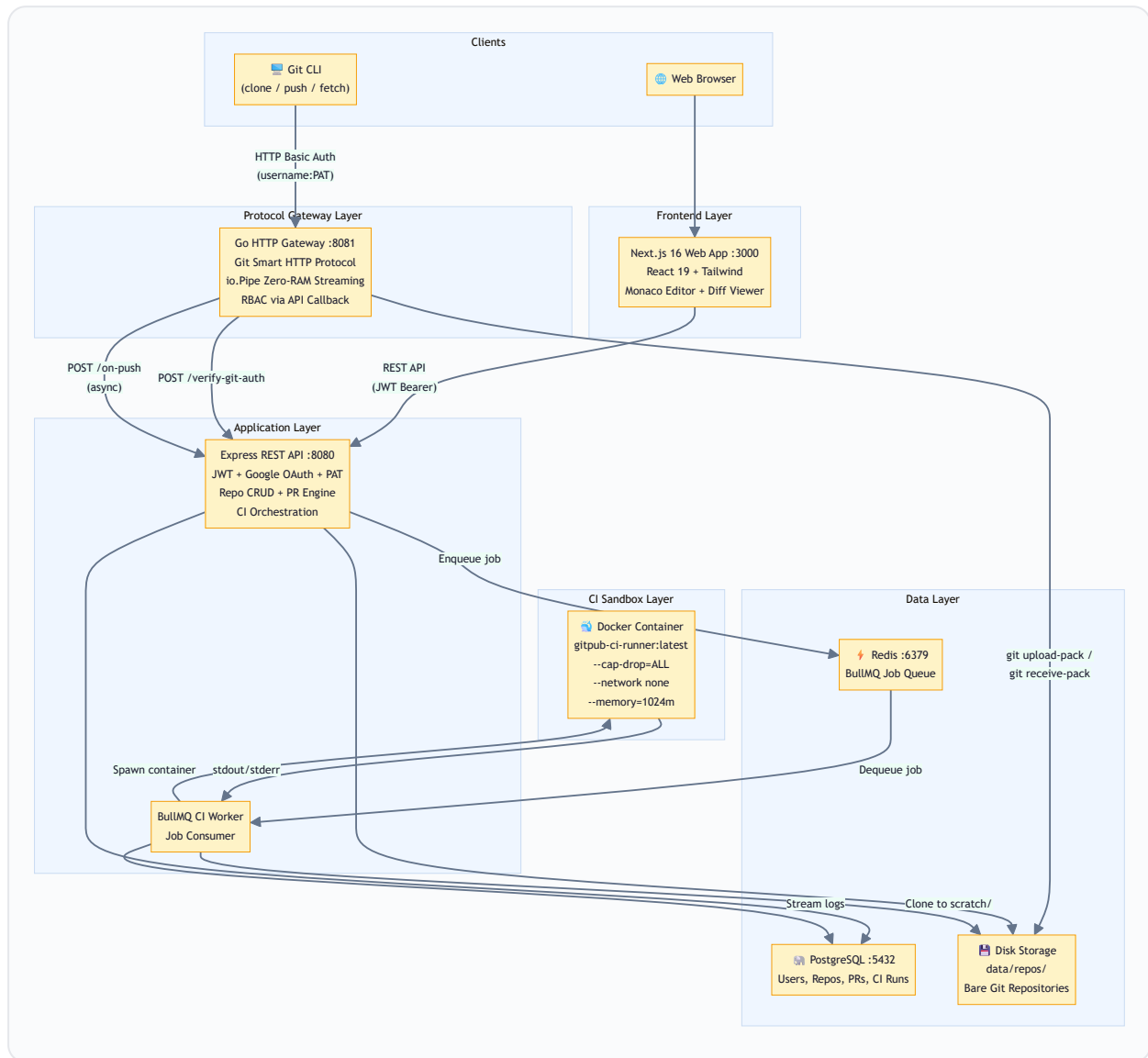
METHOD	ENDPOINT	AUTH	MAPS TO	DESCRIPTION
POST	/api/ci/internal/on-push	Internal	FR-9	Auto-trigger CI on push. Reads <code>.gitpub-ci.yml</code> .
POST	/api/ci/trigger	JWT	FR-10	Manual CI trigger <code>{ repoId, branchOrCommit }</code> .
GET	/api/ci?repoId=X	JWT	FR-11	List all CI runs for a repo.
GET	/api/ci/:id	JWT	FR-11	Get single CI run with full log.

5.7 Health Check

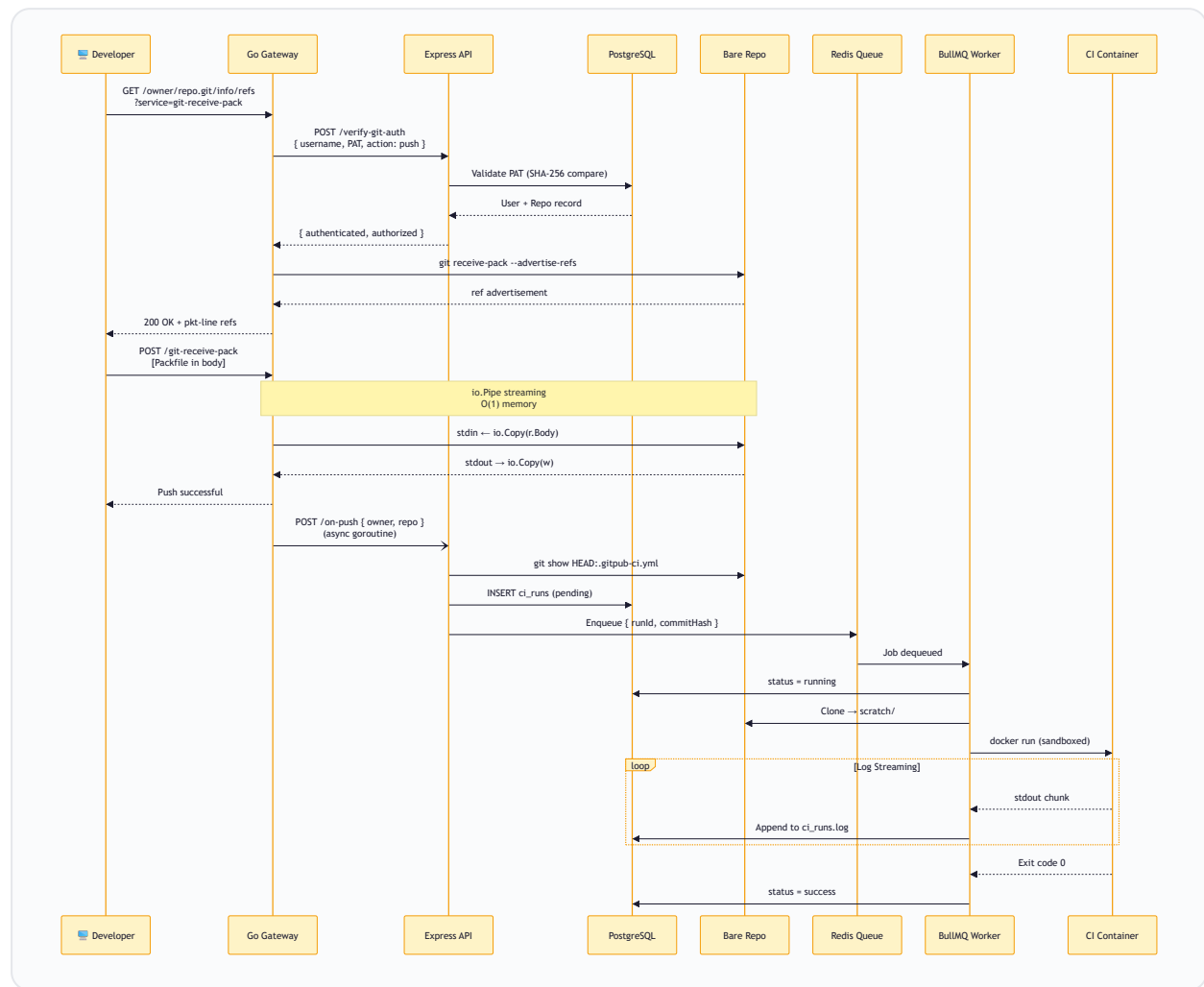
METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/health	None	Pings PostgreSQL. Returns <code>{ status: 'healthy' }</code> .

6 High-Level Design (HLD)

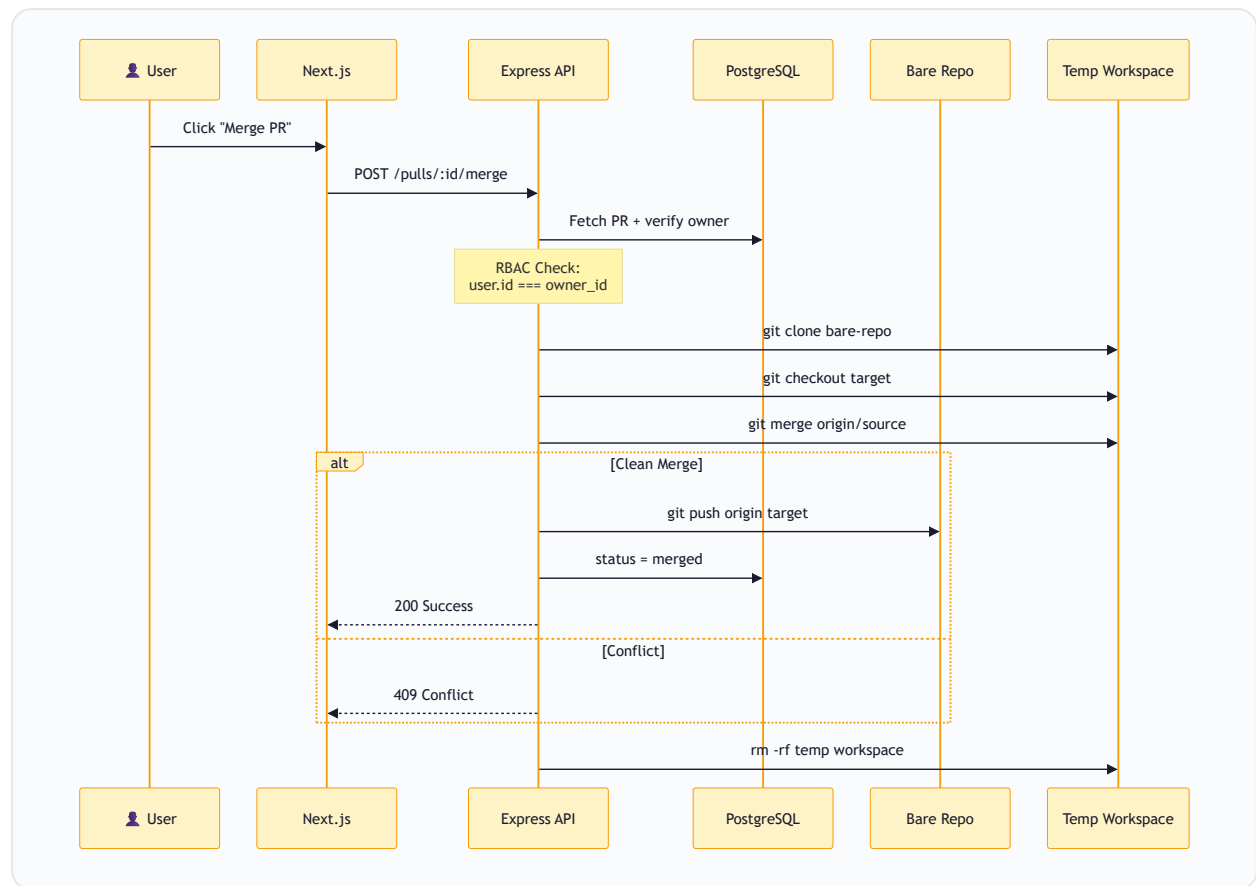
6.1 System Architecture



6.2 Git Push — End-to-End Sequence

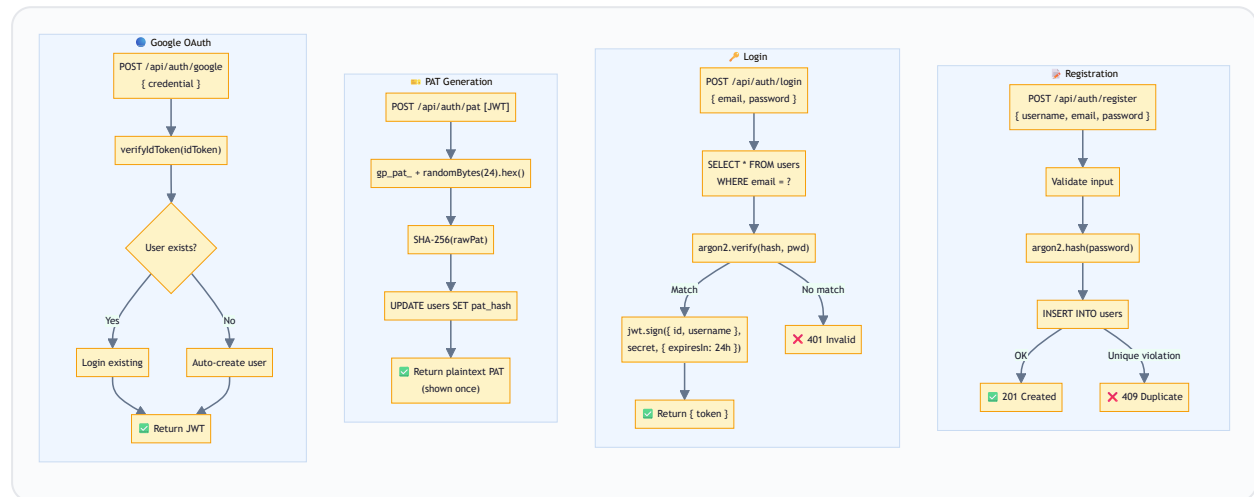


6.3 Pull Request Merge Flow



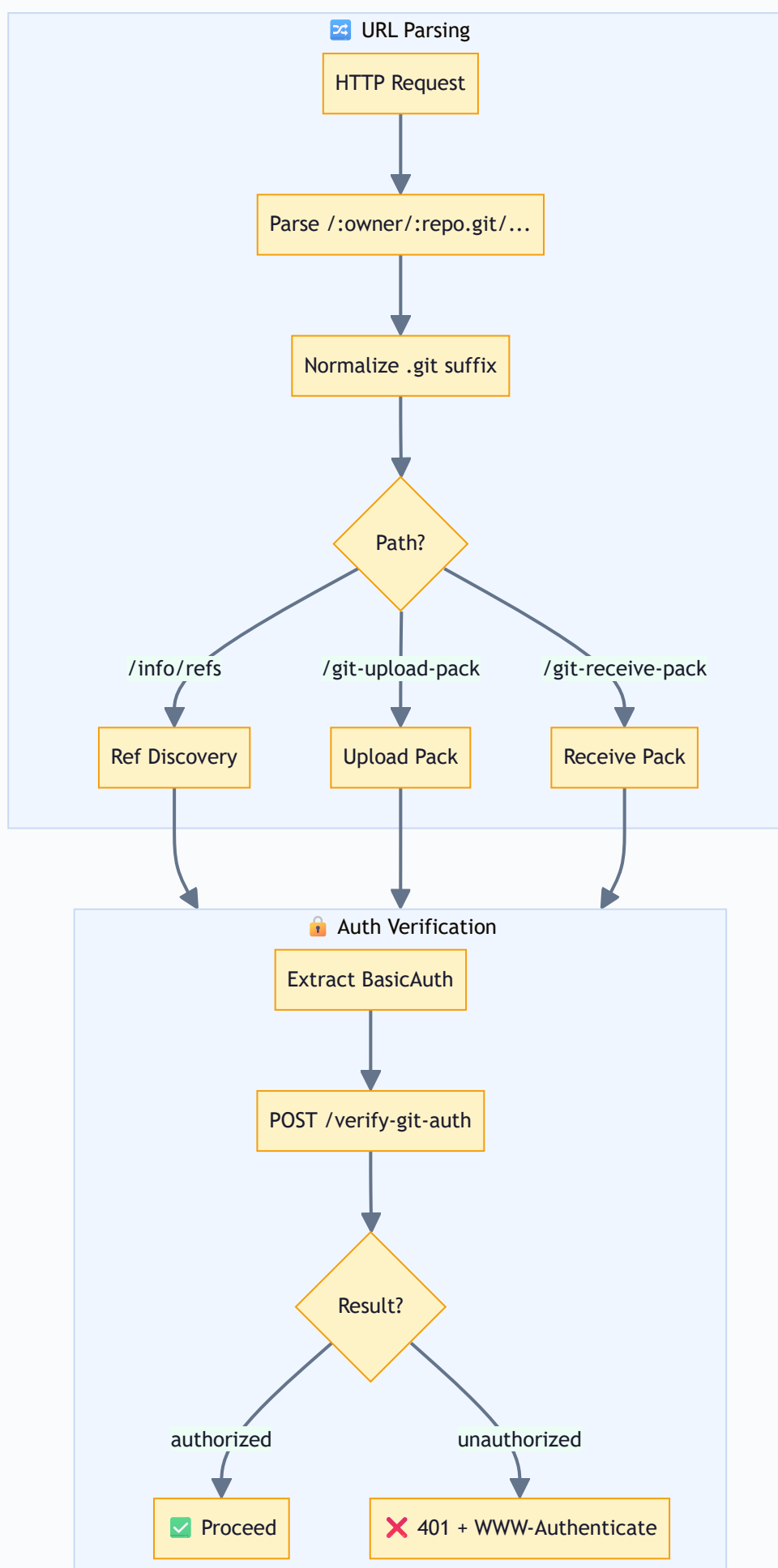
7 Low-Level Design (LLD) Deep Dive

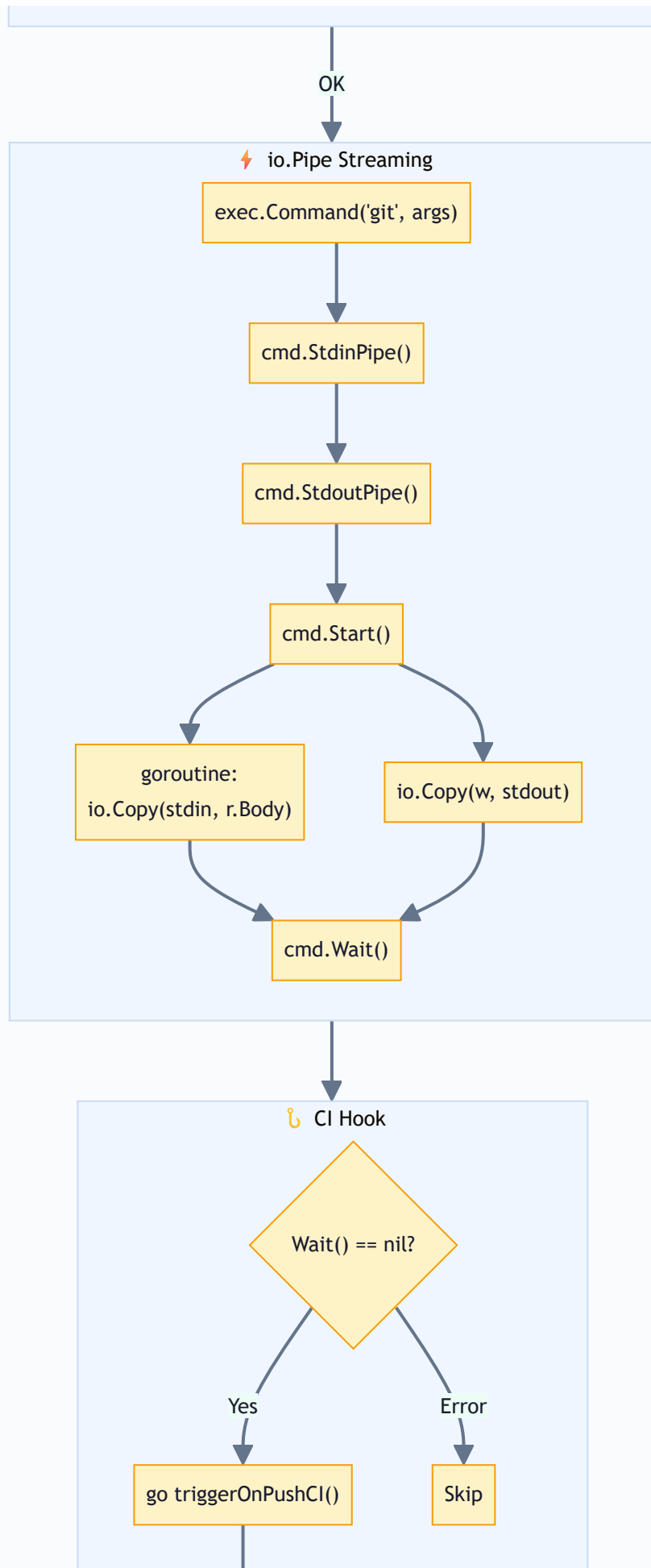
7.1 Authentication System



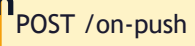
7.2 Git Protocol Gateway







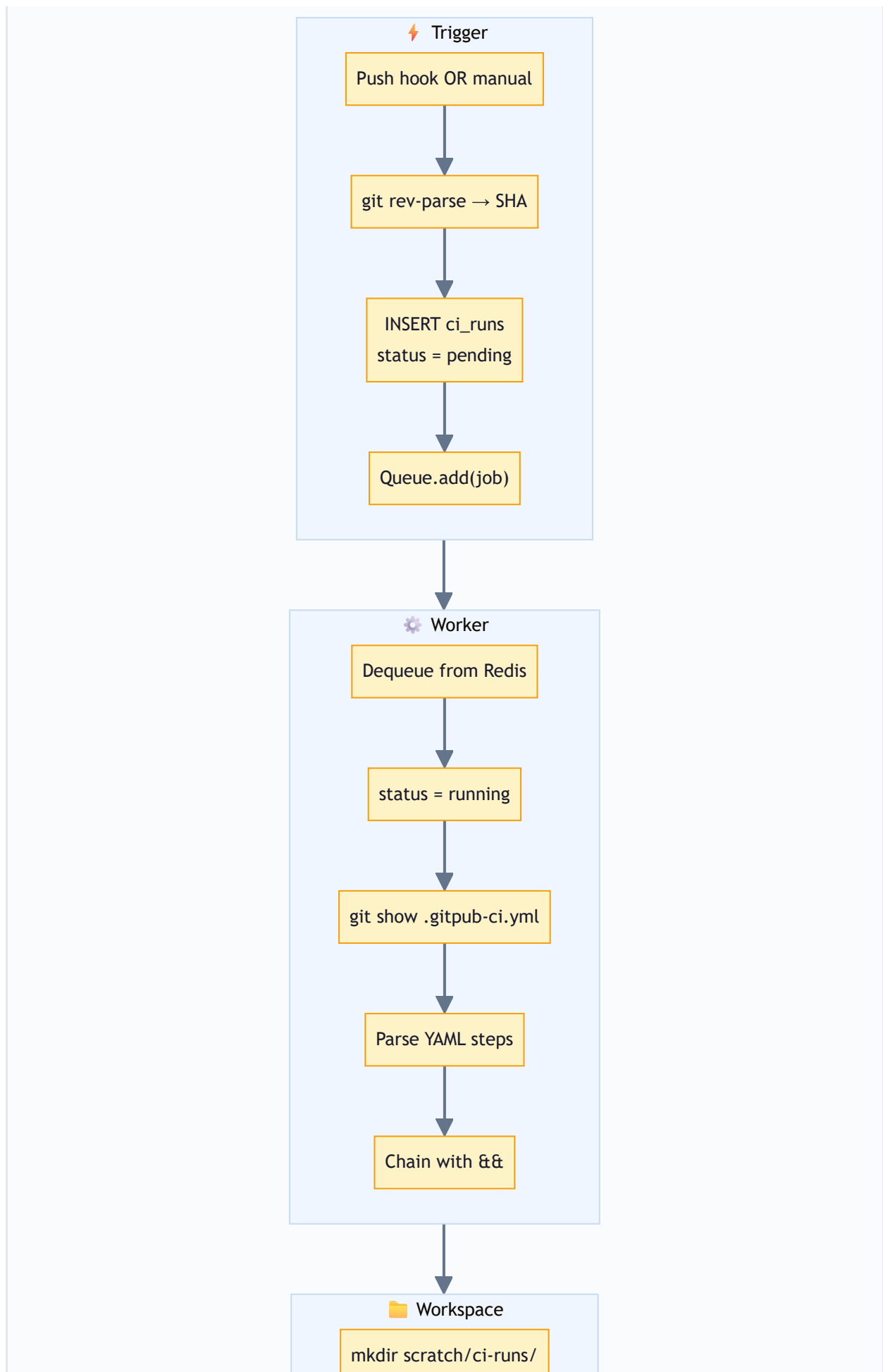
7.3 CI Pipeline Execution

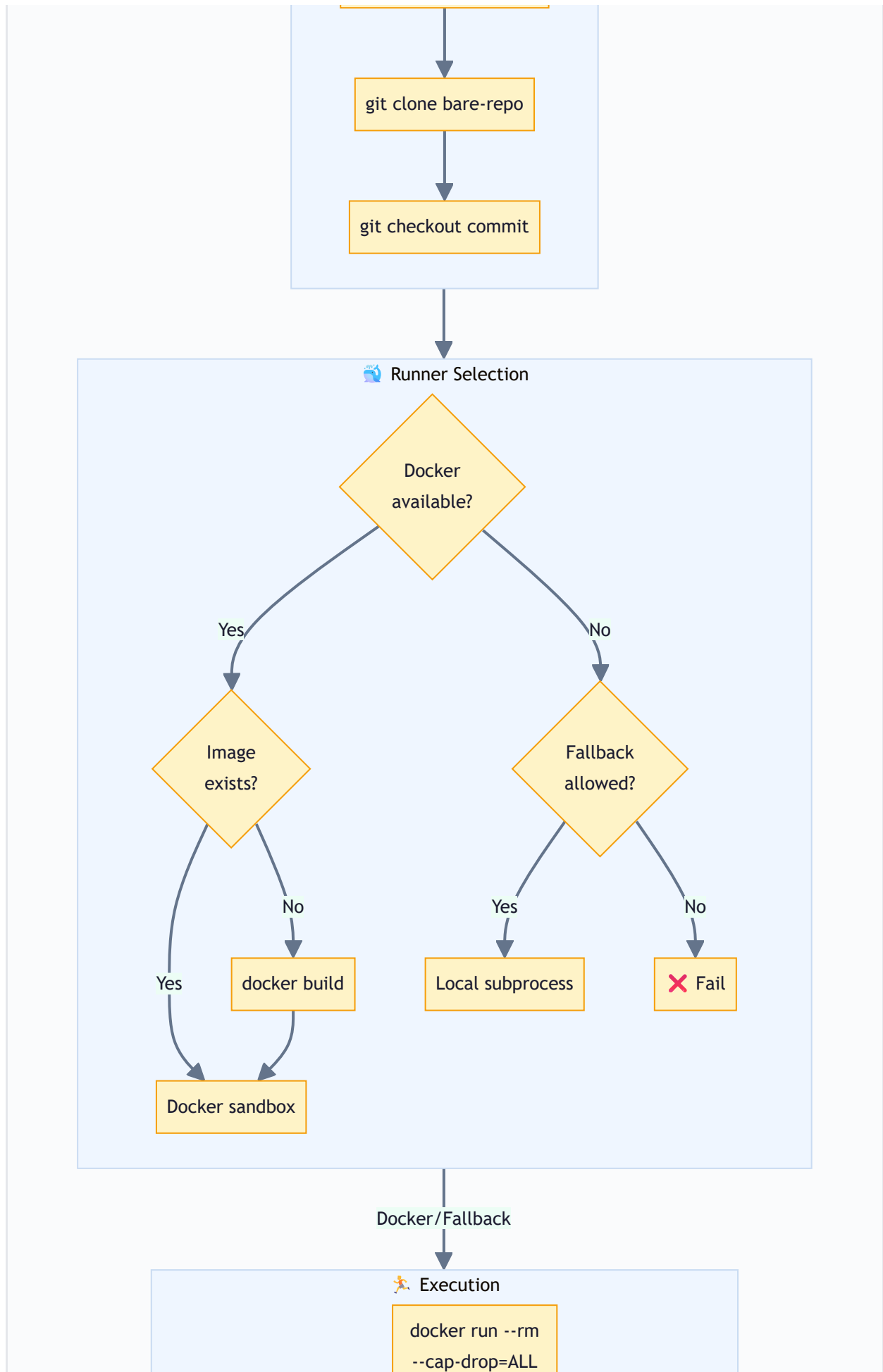


POST /on-push

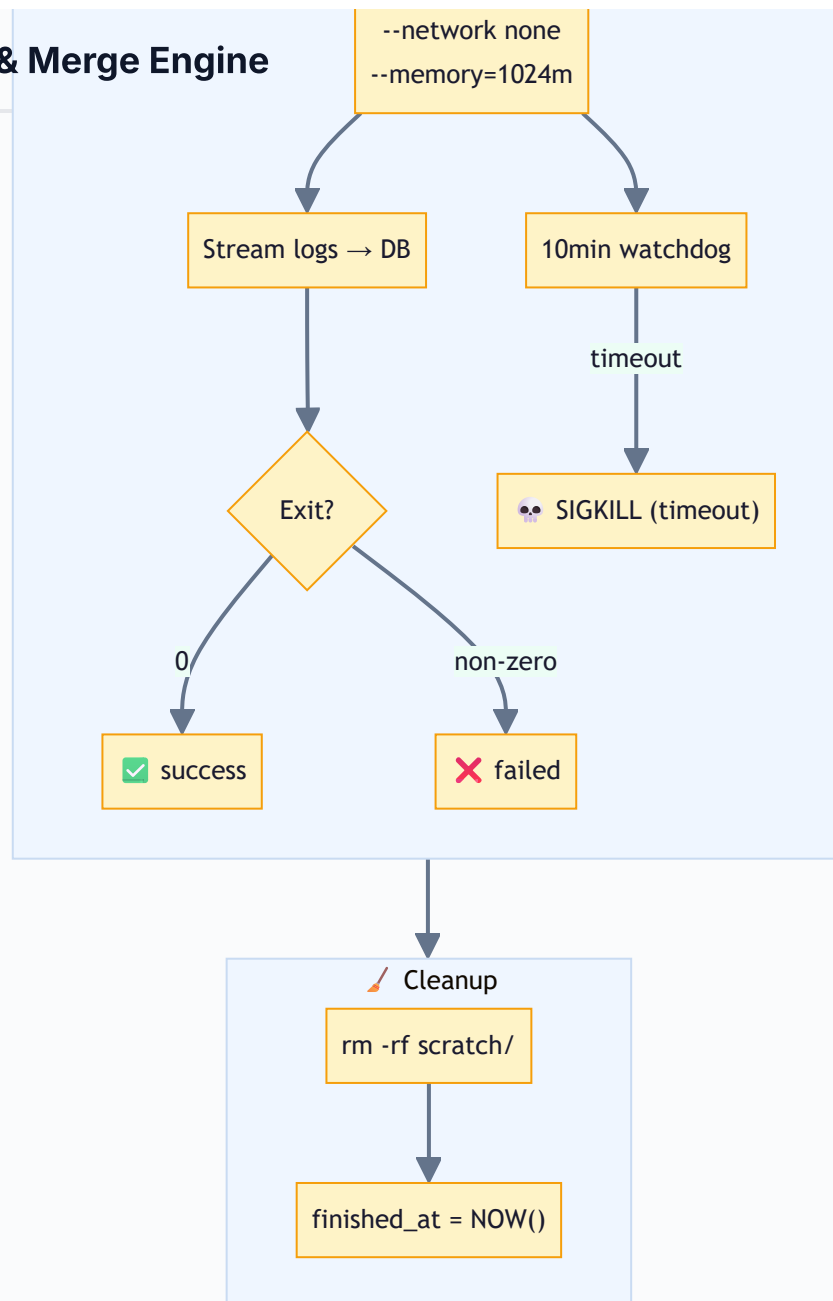
The diagram illustrates a trigger event for CI pipeline execution. A dark grey arrow points downwards from the top center of the page into a light blue rectangular box. Inside this box, there is a smaller yellow rectangular box with a black border containing the text 'POST /on-push'.



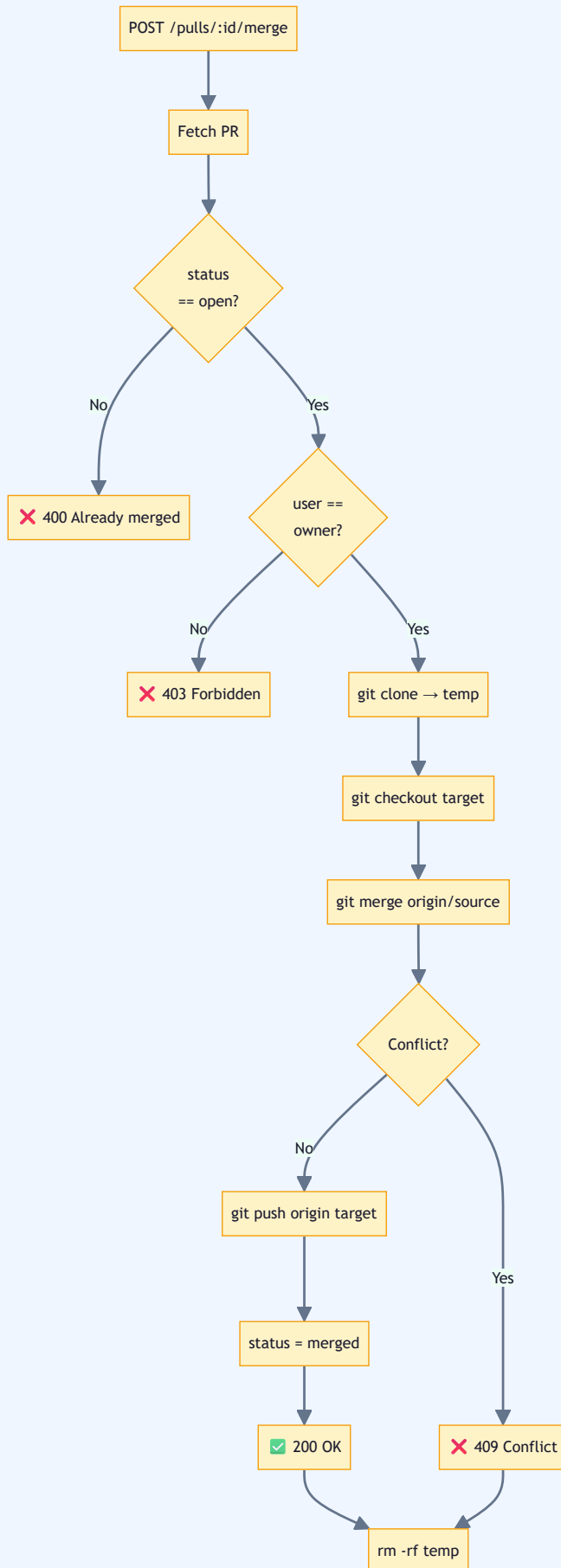




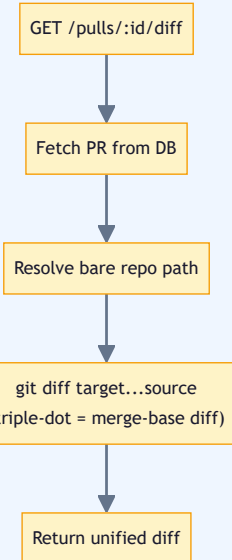
7.4 PR Diff & Merge Engine



Merge Engine



Diff Engine



💡 **Why triple-dot diff?**

`git diff A...B` computes the diff from the merge-base of A and B to B. This shows only changes introduced in the source branch, ignoring commits already in the target — exactly what a PR review needs.

💡 **Why clone for merge?**

Bare repositories have no working tree or index, so `git merge` cannot run directly on them. The system creates an ephemeral scratch clone, performs the merge, then pushes the result back.

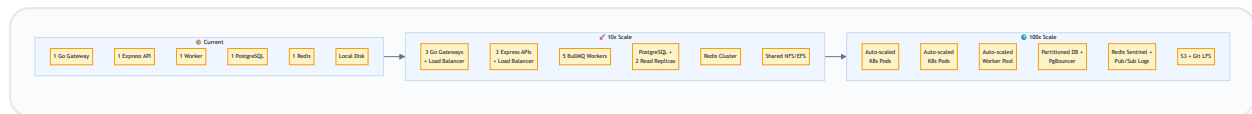
8

Database Access Patterns & Indexing

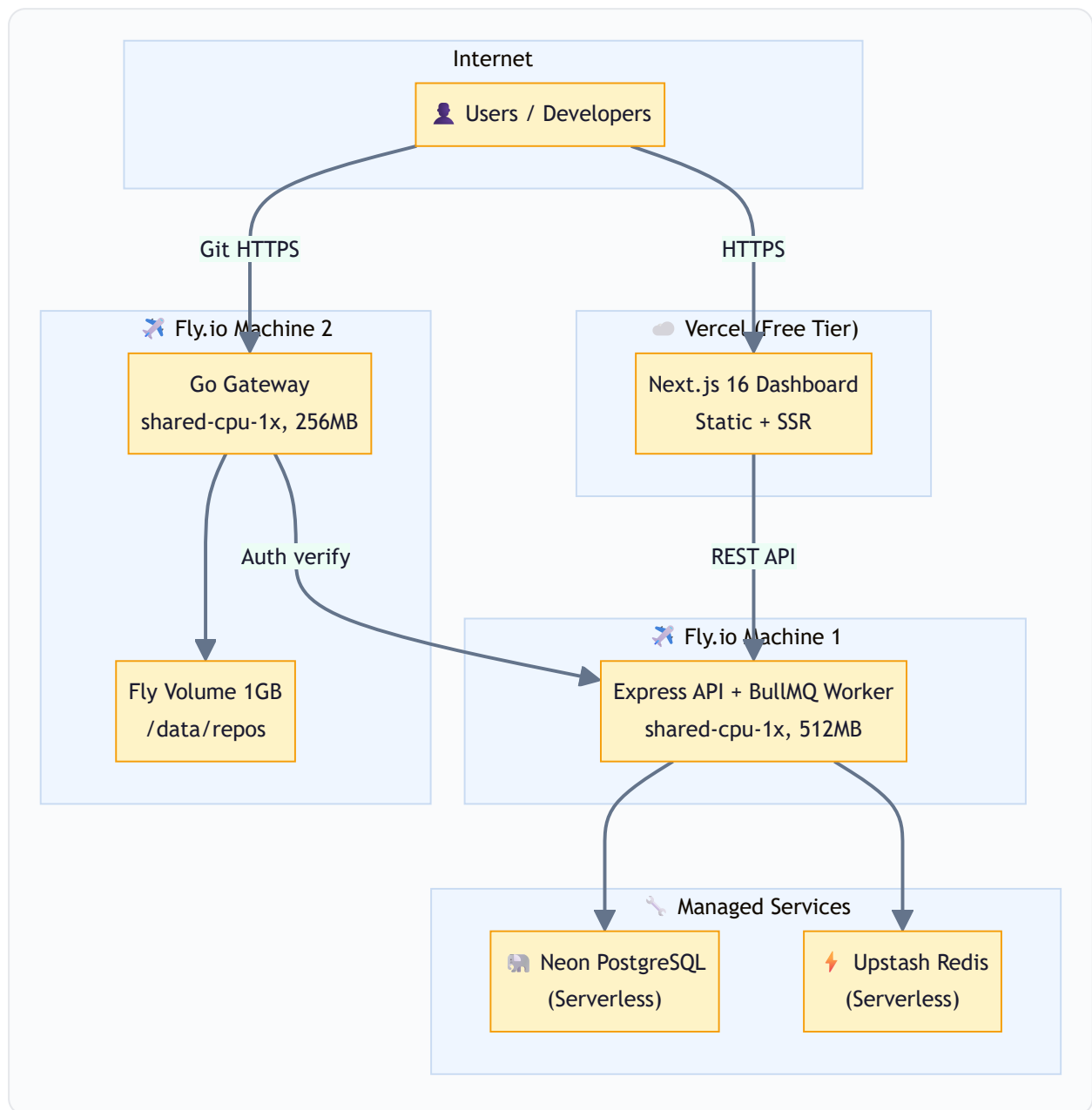
QUERY PATTERN	TABLE	INDEX	FREQUENCY
Login by email	users	UNIQUE(email) 	High
PAT validation by username	users	UNIQUE(username) 	High (every git op)
List repos by owner	repositories	(owner_id, created_at DESC)	Medium
Find repo by owner + name	repositories	UNIQUE(owner_id, name) 	High (every git op)
List PRs by repo	pull_requests	(repo_id, created_at DESC)	Medium
List CI runs by repo	ci_runs	(repo_id, created_at DESC)	High (polling)
Append CI log	ci_runs	PK lookup by id 	Very High (streaming)

9

Scaling Deep Dive — 10x / 100x



BOTTLENECK	10X SOLUTION	100X SOLUTION
Git I/O	Shared NFS volume	Shard repos via consistent hashing
CI queue	More workers	K8s HPA auto-scaling worker pool
DB writes (CI logs)	Batch appends (1s intervals)	Redis Pub/Sub → SSE. Write to DB on completion only.
API throughput	Horizontal + LB	K8s auto-scaling + PgBouncer
Log polling	Acceptable at 5s	Replace with WebSocket/SSE push



"Walk me through the architecture"

GitPub uses a **microservice architecture** with 3 core services: a **Go protocol gateway** for Git wire protocol streaming, a **Node.js/Express API** for business logic and CI orchestration, and a **Next.js frontend**. Services communicate via REST. CI jobs are dispatched through a **Redis-backed message queue** (BullMQ) and executed in **hardened Docker sandbox containers**.

"How do you handle large file uploads?"

The Go gateway uses **io.Pipe zero-RAM streaming** — the HTTP request body is piped directly to **git receive-pack**'s stdin, and stdout pipes directly to the HTTP response. Memory is $O(1)$ regardless of packfile size (even for 1GB+ pushes).

"How do you ensure security for CI?"

CI containers run with **--cap-drop=ALL** (no Linux capabilities), **--network none** (air-gapped), **--no-new-privileges**, **1GB memory cap**, **1 CPU limit**, and as **non-root user (uid 1000)**. A 10-minute watchdog timer kills hung processes. Docker socket is never mounted.

"What's your CAP theorem trade-off?"

GitPub is a **CP system**. Git refs must be linearizable — a push must be fully committed before any reader sees new refs. We use PostgreSQL (strong consistency) and prefer returning errors over serving stale data during network partitions.

"How would you scale this?"

Horizontally: Go gateways and Express APIs are stateless — scale behind a load balancer. BullMQ workers scale independently by adding more consumers. **Storage:** move to shared NFS/EFS, then shard by owner hash. **Logs:** replace polling with WebSocket/SSE and Redis Pub/Sub. **Database:** read replicas, PgBouncer, partition ci_runs table.

"What would you improve?"

1. **WebSocket/SSE** for real-time CI logs instead of polling
2. **Rate limiting** on auth endpoints (prevent brute force)
3. **Repository forking** and cross-user pull requests
4. **Branch protection rules** (require CI pass before merge)
5. **Webhook integrations** (Slack, Discord notifications)
6. **Git LFS** support for large binary assets

"Why Go for the gateway and Node.js for the API?"

Go excels at I/O-heavy, concurrent streaming workloads — goroutines give us one lightweight thread per Git connection with O(1) memory via `io.Pipe`. **Node.js** offers faster iteration for CRUD/business logic with a rich ecosystem (BullMQ, Argon2, JWT, Google Auth libraries). This polyglot approach plays to each language's strengths.

"How does the diff engine work on bare repos?"

Bare repos have no working tree, but Git can still compute diffs directly using **triple-dot notation**: `git diff target...source`. This computes the diff from the merge-base to the source branch tip, showing only new changes — exactly what PR reviews need. No temporary clone required for diffs.